

Good. Then we proceed on reasonable assumptions and you correct anything tomorrow.

Here's the deal — before I help you write the schema, let me give you the **logical model** first. The schema is just SQL. The logical model is the thinking. Get the thinking wrong, the SQL doesn't save you.

The Logical Model (Assumptions-First)

Core Entities

centers → the physical branches. Everything in the system belongs to a center. This is your multi-tenancy root.

programs → Shiksha, Sanskar, Swayam, Saksham, Sehat. A program runs *at* a center. Same program can run at multiple centers independently.

students → enrolled in a program, at a center.

users → every human who logs in. One table. Role is an attribute, not a separate table.

activities → discrete events under a program. An activity has a date range. This is also how you scope volunteers.

The Role/Access Model (Assumed)

Role	Center scope	Time-bound	Data access
admin	all centers	no	everything
teacher	assigned center(s)	no	own center students
staff	assigned center(s)	no	own center, broader than teacher
volunteer	assigned activity only	yes — custom window	only that activity's students
parent	none (student-bound)	no	only their child's data
shareholder	all centers	no	aggregated dashboards, zero PII

The critical insight: **volunteer access is scoped to an activity AND a time window**. This can't be an afterthought in the schema.

The Permission Enforcement Strategy

You have two places to enforce access: the application layer (your API code) and the database layer (Postgres Row Level Security).

For a team your size and timeline: **enforce in the API layer, design the schema to make it easy**. Postgres RLS is powerful but adds complexity when you're moving fast. You can add it later. What you must not do is design tables where center isolation requires 3 joins to verify — that's where bugs hide.

Rule: **every table that holds student data carries center_id directly**. Not through a join. Directly. This makes every query's WHERE clause obvious and auditable.

Schema — Core Tables

```
-- CENTERS
centers (
  id          UUID PRIMARY KEY,
  name        TEXT NOT NULL,
  location    TEXT,
  is_active   BOOLEAN DEFAULT true,
  created_at  TIMESTAMPTZ DEFAULT now()
)

-- PROGRAMS (Shiksha / Sanskar / Swayam / Saksham / Sehat + future)
programs (
  id          UUID PRIMARY KEY,
  code        TEXT UNIQUE NOT NULL, -- 'SWAYAM', 'SHIKSHA', etc.
  name        TEXT NOT NULL,
  age_min     INT,
  age_max     INT,
  description TEXT,
  is_active   BOOLEAN DEFAULT true
)

-- WHICH PROGRAMS RUN AT WHICH CENTER
center_programs (
  id          UUID PRIMARY KEY,
  center_id   UUID REFERENCES centers(id),
  program_id  UUID REFERENCES programs(id),
  is_active   BOOLEAN DEFAULT true,
  UNIQUE(center_id, program_id)
)
```

```

-- ALL USERS (admin, teacher, staff, volunteer, parent, shareholder)
users (
  id          UUID PRIMARY KEY,
  email       TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  full_name   TEXT NOT NULL,
  phone       TEXT,
  role        TEXT NOT NULL CHECK (role IN
    ('admin','teacher','staff','volunteer','parent','shareholder')),
  is_active   BOOLEAN DEFAULT true,
  created_at  TIMESTAMPTZ DEFAULT now()
)

-- CENTER ACCESS (teacher/staff/admin assigned to center, time-optional)
user_center_assignments (
  id          UUID PRIMARY KEY,
  user_id     UUID REFERENCES users(id),
  center_id   UUID REFERENCES centers(id),
  valid_from  DATE NOT NULL DEFAULT CURRENT_DATE,
  valid_until DATE, -- NULL = permanent
  created_by  UUID REFERENCES users(id),
  UNIQUE(user_id, center_id)
)

-- ACTIVITIES (events under a program at a center)
activities (
  id          UUID PRIMARY KEY,
  center_id   UUID REFERENCES centers(id),
  program_id  UUID REFERENCES programs(id),
  name        TEXT NOT NULL,
  description TEXT,
  start_date  DATE,
  end_date    DATE,
  created_by  UUID REFERENCES users(id),
  created_at  TIMESTAMPTZ DEFAULT now()
)

-- VOLUNTEER ACCESS (scoped to a specific activity + time window)
user_activity_assignments (
  id          UUID PRIMARY KEY,
  user_id     UUID REFERENCES users(id),
  activity_id UUID REFERENCES activities(id),
  valid_from  DATE NOT NULL,
  valid_until DATE,
  created_by  UUID REFERENCES users(id)
)

```

```

)

-- STUDENTS
students (
  id          UUID PRIMARY KEY,
  center_id   UUID REFERENCES centers(id), -- direct, always
  program_id  UUID REFERENCES programs(id),
  full_name   TEXT NOT NULL,
  dob         DATE,
  gender      TEXT CHECK (gender IN ('male','female','other')),
  guardian_name TEXT,
  guardian_phone TEXT,
  enrollment_date DATE DEFAULT CURRENT_DATE,
  is_active   BOOLEAN DEFAULT true,
  created_at  TIMESTAMPTZ DEFAULT now()
)

-- PARENT → STUDENT LINK
parent_student (
  id          UUID PRIMARY KEY,
  parent_user_id UUID REFERENCES users(id),
  student_id  UUID REFERENCES students(id),
  UNIQUE(parent_user_id, student_id)
)

```

Schema — Tracking Tables

```

-- ATTENDANCE SESSIONS (a class/session on a date)
attendance_sessions (
  id          UUID PRIMARY KEY,
  center_id   UUID REFERENCES centers(id),
  program_id  UUID REFERENCES programs(id),
  activity_id UUID REFERENCES activities(id), -- nullable if regular session
  session_date DATE NOT NULL,
  created_by  UUID REFERENCES users(id),
  created_at  TIMESTAMPTZ DEFAULT now()
)

-- ATTENDANCE RECORDS
attendance_records (
  id          UUID PRIMARY KEY,
  session_id  UUID REFERENCES attendance_sessions(id),
  student_id  UUID REFERENCES students(id),
  center_id   UUID REFERENCES centers(id), -- denormalized intentionally

```

```

        status      TEXT CHECK (status IN ('present','absent','late')),
        remarks     TEXT
    )

-- EXAMS (Baseline / Endline per program per year)
exams (
    id              UUID PRIMARY KEY,
    center_id       UUID REFERENCES centers(id),
    program_id      UUID REFERENCES programs(id),
    exam_type       TEXT CHECK (exam_type IN ('baseline','endline')),
    academic_year   TEXT NOT NULL, -- '2024-25'
    exam_date       DATE,
    created_by      UUID REFERENCES users(id)
)

-- EXAM SCORES
exam_scores (
    id              UUID PRIMARY KEY,
    exam_id         UUID REFERENCES exams(id),
    student_id      UUID REFERENCES students(id),
    center_id       UUID REFERENCES centers(id), -- denormalized
    subject         TEXT NOT NULL, -- 'english','maths','science'
    marks           NUMERIC(5,2),
    max_marks       NUMERIC(5,2) DEFAULT 50,
    remarks         TEXT
)

-- FLEXIBLE FORMS (JSONB - add new form types without schema changes)
form_templates (
    id              UUID PRIMARY KEY,
    form_type       TEXT NOT NULL, -- 'student_meeting', 'parent_meeting', 'activity'
    name            TEXT NOT NULL,
    schema          JSONB NOT NULL, -- field definitions
    is_active       BOOLEAN DEFAULT true,
    created_at      TIMESTAMPTZ DEFAULT now()
)

form_submissions (
    id              UUID PRIMARY KEY,
    template_id     UUID REFERENCES form_templates(id),
    student_id      UUID REFERENCES students(id),
    center_id       UUID REFERENCES centers(id),
    submitted_by    UUID REFERENCES users(id),
    data            JSONB NOT NULL,
    created_at      TIMESTAMPTZ DEFAULT now()
)

```

The Three Design Decisions Worth Explaining to Your Team

1. UUID over serial integers. You're building multi-center. When you eventually merge data, sync, or export — sequential integers collide. UUIDs don't. Use `gen_random_uuid()` in Postgres.

2. center_id is denormalized into attendance and exam score records. Technically you could derive it via join. But putting it directly in the row means every access-control check is a single WHERE clause, not a subquery. This is the correct tradeoff.

3. JSONB for forms, typed tables for everything else. Forms are inherently variable. Exam scores, attendance — they're structured and you'll be aggregating them. Keep structured data in proper columns, flexible data in JSONB. Don't mix the two philosophies.

What to Clarify Tomorrow

- How many centers currently exist, and do they all run Swayam or different programs per center?
- Is attendance per-student per-session, or per-student per-day (some NGOs do daily, some per-activity)?
- Exam subjects — are they fixed for Swayam (English/Maths/Science) or configurable?
- Do parents get a login in Phase 1 or just Phase 2?
- Who is the “admin” — one person for the whole NGO, or one admin per center?

Those answers will either confirm this schema or require adjustments to 2-3 tables. Everything else we can safely assume.

Ready to move to indexing strategy, or do you want to stress-test this schema first?